

GraphFVD: Property Graph-based Fine-grained Vulnerability Detection

Miaomiao Shao, Yuxin Ding*, Jing Cao, Yilin Li

^a*School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China*

Abstract

Deep learning technology can automatically extract features from software source code, making it widely used for detecting software vulnerabilities. Most existing deep learning-based approaches rely on whole functions or sequence-level program slices to identify vulnerabilities. However, these approaches often struggle to capture comprehensive vulnerability semantics, leading to high false positive rates and false negative rates. In this paper, we propose GraphFVD, a novel property graph-based fine-grained vulnerability detection approach. Our approach extracts property graph-based slices from the Code Property Graph and introduces a Hierarchical Attention Graph Convolutional Network to learn graph embeddings. GraphFVD provides a fine-grained code representation that captures syntax, control flow, data flow, and the natural sequential order of source code relevant to vulnerabilities. We evaluate the effectiveness of our approach on two real-world vulnerability datasets. Experimental results demonstrate that our approach outperforms existing state-of-the-art vulnerability detection methods on both datasets.

Keywords:

Vulnerability detection, Program slicing, Deep learning, Hierarchical attention, Relational Graph Convolutional Network

1. Introduction

In the software development life cycle, design flaws or coding errors can inevitably introduce vulnerabilities. When discovered and exploited by cyber attackers, these vulnerabilities pose serious threats to system security. Despite focused efforts from academia and industry to enhance software security, the number of vulnerabilities continues to grow rapidly. According to data from the CVE security vulnerability database¹, 29,065 new vulnerabilities were reported in 2023—a significant increase of 15.88% compared to 2022. The features of different vulnerability types vary greatly, and even the same type of vulnerability can take diverse forms across different files. This diversity makes building a model capable of detecting multiple vulnerabilities a significant challenge. Traditional vulnerability detection methods are typically limited to detecting only a few vulnerability types and heavily rely on rules and features manually defined by security experts. This manual process requires considerable human effort and is prone to errors.

In recent years, deep learning technology has achieved remarkable success in computer vision and natural language processing, largely due to its capacity for automatic feature extraction. Consequently, many studies have explored using deep learning techniques to analyze programs and identify vulnerabilities in source code. Previous research shows that deep learning-based detection methods outperform traditional program analysis-based approaches [1, 2, 3, 4, 5]. Among previous deep learning-based methods, some studies treat the whole function as the detection object [6, 7]. However, in a vulnerable function, only a few statements may relate directly to the vulnerability, making these function-based methods coarse-grained and limiting their detection accuracy. Recently, slice-based approaches have gained significant interest in vulnerability detection [1, 2, 8, 9, 10]. These approaches typically generate sequence-level program slices from a Program Dependency Graph (PDG), where each slice consists of multiple semantically

*Corresponding author

Email address: yxding@hit.edu.cn (Yuxin Ding)

¹<https://www.cvedetails.com/>

related statements. The program slice is then converted into a token sequence, allowing natural language processing techniques to be applied for vulnerability detection.

However, existing slice-based detection methods still have notable limitations. First, it is well known that code is more structured and logical than natural language. Therefore, sequence-level code representations cannot effectively reflect syntax and semantics of the code structure. These limitations restrict the capability of the model to detect vulnerabilities accurately. Second, although PDG can capture both control and data dependencies in source code, it does not capture the abstract syntax or the natural sequential order of statements. Consequently, vulnerability detection methods based on PDG struggle to identify certain vulnerabilities, such as memory disclosure and integer overflows [11]. Furthermore, most previous vulnerability detection models are trained on synthetic datasets, which leads to reduced performance when applied to real-world open-source projects [2].

To this end, This paper presents a novel vulnerability detection approach that extracts graph-level program slices from the Code Property Graph (CPG) and introduces a Hierarchical Attention Graph Convolutional Network (HierGCN) to learn graph embeddings. The CPG integrates the Abstract Syntax Tree (AST), Control Flow Graph (CFG), and Program Dependency Graph (PDG), enabling it to capture diverse vulnerability characteristics related to syntax, control flow, and data flow. By extracting property graph-based slices from the CPG, our approach more accurately captures the essential characteristics of vulnerable code. We evaluate the effectiveness of our approach using two real-world vulnerability datasets. The datasets and source code presented in this paper are available online².

Our main contributions are summarized as follows:

- We propose a novel fine-grained code representation that extracts graph-level program slices (i.e., Property Graph-based Vulnerability Candidates or PrVCs) from the Code Property Graph. The PrVC combines abstract syntax, control flow, data flow and the sequential order of the code, allowing it to capture comprehensive and precise program semantics.
- We design a Hierarchical Attention Graph Convolutional Network (HierGCN) to learn the embeddings of PrVCs and predict the vulnerability. HierGCN introduces hierarchical attention in Relational Graph Convolutional Networks (RGCN), incorporating both local attention and global attention. This attention mechanism enables us to fuse the features of different tokens and nodes based on their importance.
- We evaluate the effectiveness of our approach GraphFVD using two real-world vulnerability datasets. The experimental results demonstrate that GraphFVD outperforms state-of-the-art vulnerability detection techniques.

The rest of this article is organized as follows: Section 2 introduces the related work. Section 3 shows the details of our GraphFVD. In Section 4 and Section 5, we describe the experimental setup and results, respectively. In Section 6, we discuss the limitation of GraphFVD. Finally, Section 7 concludes the paper.

2. Related Work

2.1. Program Analysis-based Approaches

Program analysis-based approaches can be broadly classified into three categories: symbolic execution-based methods, rule-based methods, and code similarity-based methods. Symbolic execution-based methods combine symbolic execution with constraint solving to identify vulnerabilities [12, 13]. By using symbolic execution, the values of variables in the program are represented as computational expressions composed of symbolic values and constants. Some vulnerability patterns are identified when the values of specific variables do not satisfy the corresponding constraints. However, since constraint solving cannot address all types of constraints, the analysis performance of symbolic execution may be limited. Furthermore, due to the high computational overhead, symbolic execution-based methods are not suitable for large-scale code vulnerability analysis.

Rule-based approaches leverage expert-crafted rules to identify vulnerabilities, such as Flawfinder [14] and Checkmarx [15]. However, these systems often exhibit high false positive rates and high false negative rates. The process of manually defining features is not only time-consuming but also task-specific, resulting in poor generalization ability.

²<https://github.com/VulDet/GraphFVD>

Code similarity-based methods transform source code into an easily analyzable intermediate code representation and use a similarity matching algorithm to achieve vulnerability detection. VulPecker [16] is a typical similarity-based vulnerability detector for software source code. Sun et al. [17] exploited a Siamese model to identify the similarity among different versions of the same function and the distinction between the vulnerable code and the patch code. However, similarity-based methods can only detect vulnerabilities related to code duplication.

2.2. Deep Learning-based Vulnerability Detection

To alleviate the dependence on manually extracted features, researchers attempt to automatically learn program features using deep learning techniques. Based on the detection granularity, existing deep learning-based detection methods can be divided into function-based methods and slice-based methods. Function-based methods typically detect vulnerabilities based on individual functions. Many of these methods extract graph-level representations from the function, such as AST and CFG, to analyze the semantics of the vulnerability code [4, 5, 18, 19]. Li et al. presented VulDeePecker [20], SySeVR [1], and VulDeeLocator [2] to detect vulnerable code based on program slices, which offer a more fine-grained approach than functions. Subsequently, an increasing number of slice-based detection methods have been proposed [8, 9, 10, 21, 22]. Additionally, Zou et al. [23] introduced a deep learning-based approach for vulnerability classification. Significantly, some recent studies [2, 24] have further refined the detection granularity of vulnerable code to the statement level, which can locate vulnerable statements in program slices. However, the localization accuracy of these methods needs to be improved.

Different from the above methods, ReGVD [25] expanded the source code into a token sequence to build a graph, and then employed GNN to learn the graph embedding. Cheng et al. [26] proposed to exploit program dependency paths to detect vulnerabilities. This approach employs contrastive learning framework to instruct the path selection process, thereby reducing the number of value-flow paths. Furthermore, Wu et al. [27] proposed a vulnerability detection method that converts the source code of a function into an image, where a CNN is employed to classify the images for detecting vulnerabilities. Fu and Tantithamthayorn [28] proposed a Transformer-based statement level vulnerability detection method called LineVul. Yuan et al. [29] constructed a behavior graph by extracting abstract function behaviors and used it to develop a deep learning-based vulnerability detection method. DEEPVD [30] leverages three types of class-separation features at different abstraction levels: statement type, Post-Dominance Tree, and Exception Flow Graph. TrVD [31] is a vulnerability detection method based on AST decomposition. It introduces a novel decomposition algorithm to decompose the AST of a code fragment into an ordered set of sub-trees with size and depth constraints, allowing the effective capture of sub-tree semantics through a carefully designed tree neural network. Recently, Large Language Models (LLMs) have gained significant attention due to their remarkable capabilities, and some studies are exploring their use in vulnerability detection. Zhang et al. [32] evaluated the performance of software vulnerability detection using ChatGPT with various prompt designs. Lu et al. [33] proposed a new vulnerability detection method, GRACE, which combines graph structure information in code with contextual learning to enable LLM-based software vulnerability detection.

In this paper, we propose a property graph-based fine-grained vulnerability detection framework. Unlike previous works, our approach aims to extract graph-level program slices from Code Property Graph to capture comprehensive fine-grained semantics of vulnerabilities. Additionally, our method presents a novel graph learning framework, namely Hierarchical Attention Graph Convolutional Network.

3. The GraphFVD Framework for Fine-grained Vulnerability Detection

3.1. Framework Overview

In this paper, we propose a novel property Graph-based Fine-grained Vulnerability Detection approach (GraphFVD). Fig. 1 presents an overview of the approach, which consists of two stages: PrVC generation and vulnerability prediction.

PrVC Generation. This stage consists of two sub-stages: (i) Slicing Entry Point extraction. In this step, we identify Slicing Entry Points (SEPs) from the Code Property Graph (CPG), where each SEP serves as the starting node for extracting key characteristics of the program. (ii) Property graph-based program slicing. To enable fine-grained vulnerability detection, we design a novel program slicing method that traverses the CPG from each SEP to generate property graph-based program slices (i.e., PrVCs).

Fine-grained Vulnerability Detection. The vulnerability prediction stage also includes three sub-stages: (i) Graph learning via a Hierarchical Attention Graph Convolutional Network. Incorporating hierarchical attention into Relational Graph Convolutional Networks (RGCN), which integrates both local attention and global attention. Local attention aggregates the representations of all tokens on each node to obtain embedding vectors for nodes in PrVCs, while g attention assigns varying weights to different nodes, generating embeddings for PrVCs. (iii) Vulnerability Prediction. Using a Multi-Layer Perceptron (MLP) to classify samples as either vulnerable or non-vulnerable.

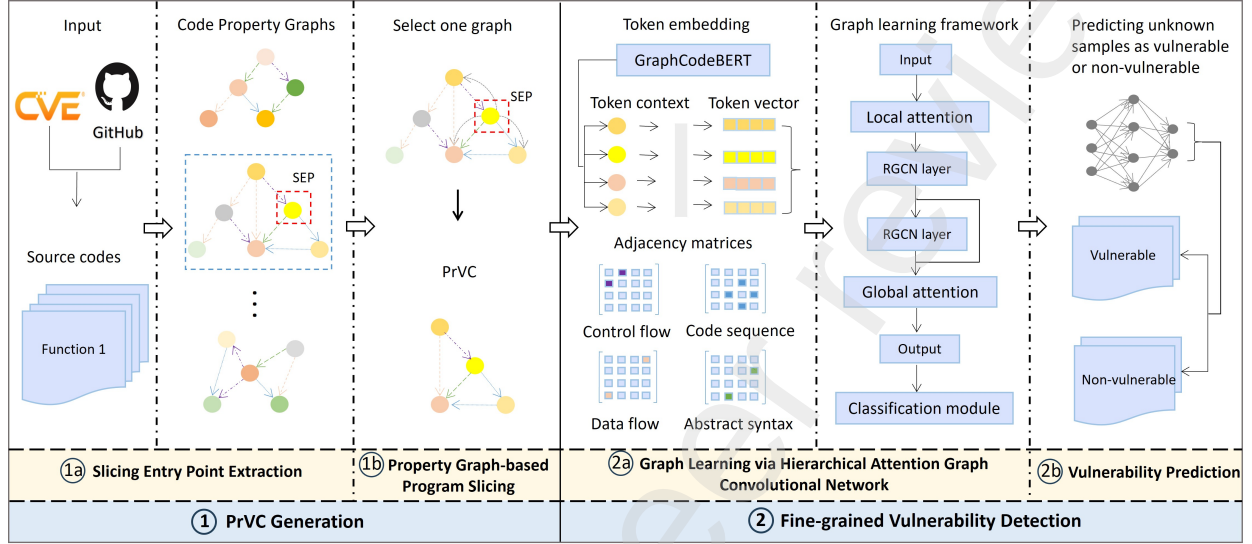


Fig. 1. Overview of GraphFVD

3.2. PrVC Generation

In this section, we present a program slicing method that extracts graph-level program slices, denoted as PrVCs, from the Code Property Graph. The PrVC captures various program semantics, including control flow, data flow, abstract syntax, and natural sequential order of the source code. This comprehensive code representation enables the extraction of both structural and unstructured features of vulnerable code, enhancing the effectiveness of vulnerability detection. Compared to functional-level code representations, the PrVC provides a more fine-grained representation, making it suitable for training a highly precise vulnerability detection model. The definition of a PrVC is provided below.

Definition 1. (*Property Graph-based Vulnerability Candidate or PrVC*). Given a set of functions $F = \{f_1, f_2, \dots, f_n\}$ and a Sling Entry Point $n_{i,j}$ in the Code Property Graph $G_i = (V_i, E_i)$ of function f_i ($1 \leq i \leq n$), the PrVC corresponding to the SEP $n_{i,j}$ is denoted as $G_i^j = (V_i^j, E_i^j)$. Each node $v \in V_i^j$ has two characteristics: code and type. The code accommodates the source code represented by v , which may be a complete statement or multiple consecutive tokens. The type of v represents its specific type, such as *Identifier*, *Parameter* and *IfStatement*. E_i is a set of directed edges, where each edge denotes abstract syntax, data flow, control flow or natural sequential order of the code between two nodes.

We start by selecting SEPs from the Code Property Graph and applying program slicing techniques to identify nodes that are semantically related to these SEPs. This approach aims to filter out redundant nodes that do not directly relate to vulnerabilities within the CPG. The generation of PrVCs is divided into two main steps: (1) Slicing Entry Points extraction and (2) Property graph-based program slicing. The specific implementation of each step is detailed below.

3.2.1. Slicing Entry Point Extraction

Thorough the analysis of numerous vulnerable code samples, we observe that the majority of vulnerabilities are closed related to function calls, sensitive variables, arithmetic operators, and if-condition statements. Based on these observations, we select Slicing Entry Points (SEPs) by matching these types of code characteristics. Specifically, we selected a total of 1,506 function calls, which were identified using several static detectors [14, 15]. Sensitive variables include pointer variables and array variables. Improper usage of sensitive variables often leads to vulnerabilities such as null pointer dereference, double free, and buffer overflow [14]. For example, as shown in Fig. 2a, if a platform-specific PMU driver is not registered, the pointer variable *ppmu* is set to null, and accessing its members triggers a null pointer dereference vulnerability. This vulnerability can be mitigated by checking whether *ppmu* is null before access. Additionally, the improper usage of arithmetic operators (e.g., addition and multiplication) is often associated with integer overflow vulnerabilities.

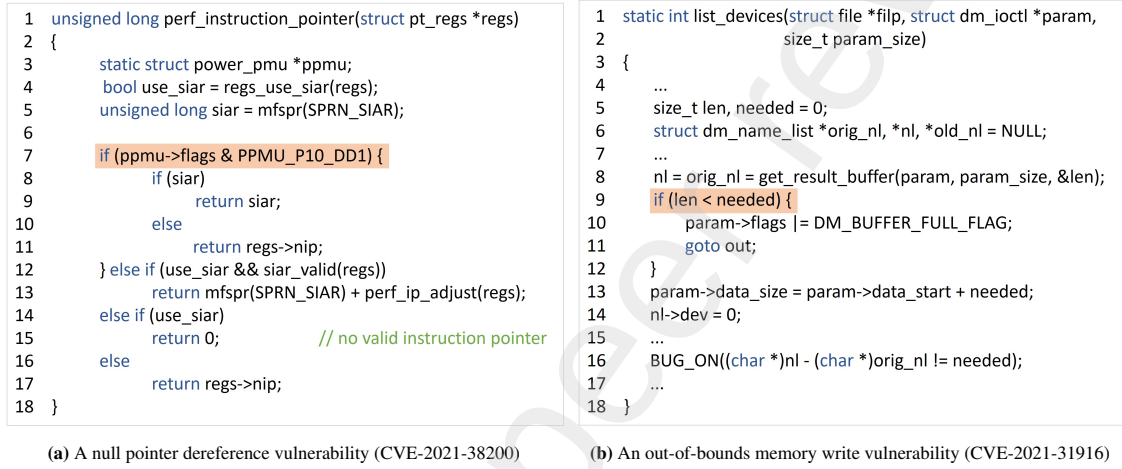


Fig. 2. Two examples of vulnerabilities in the Linux Kernel project

Furthermore, upon examining a significant number of vulnerable code instances, we observed that certain vulnerabilities are linked to if-condition statements. For example, in Fig. 2b, when there are no *dm* devices, the variable *needed* is set to zero, while the variable *len* is updated within the *get_result_buffer* function. The *dm_ioctl* struct requires eight bytes for storage. In this scenario, the if-condition statement at line 9 evaluates to false, causing the program to jump to line 13. This can result in an out-of-bounds write, as *len* could be less than eight. We can address this vulnerability by modifying the if-condition statement at line 9.

To explain the detailed process of extracting SEPs from the CPG, we present the CPG of the vulnerable function *perf_instruction_pointer* in Fig. 2a. Each blue box represents a statement in the source code, while the purple dashed lines denote the control dependency edges. The orange boxes correspond to AST nodes, which are connected by green dashed lines, representing abstract syntax edges. Additionally, each yellow dashed line illustrates the data flow from one node to another node.

We can distinguish different nodes based on two characteristics: type and code. By matching the defined code characteristics, we can extract four types of SEPs. Specifically, each SEP satisfies one of the following conditions: 1) The type is *Function* and the code corresponds to one of the 1,506 function calls; 2) The type is *IdentifierDeclStatement* or *Parameter*, and the code contains '=' and ('*' or '['); 3) The type is *ExpressionStatement* and the code contains '=' and matches an expression; 4) The type is *IfStatement* or *Parameter* and the code contains 'if'. To illustrate the SEP identification process, we take sensitive variables as an example. For sensitive variables, we initiate a query in the CPG to select all nodes of type *IdentifierDeclStatement* or *Parameter*. For each queried node, we check whether its code contains '=', and if so, we extract the tokens preceding it. We further check whether these extracted tokens contain either '*' or '['. If they do, we add this node to the set of SEPs. In Fig. 3, one of the SEPs extracted from the CPG is highlighted (i.e., Node 1, outlined in a red rectangle).

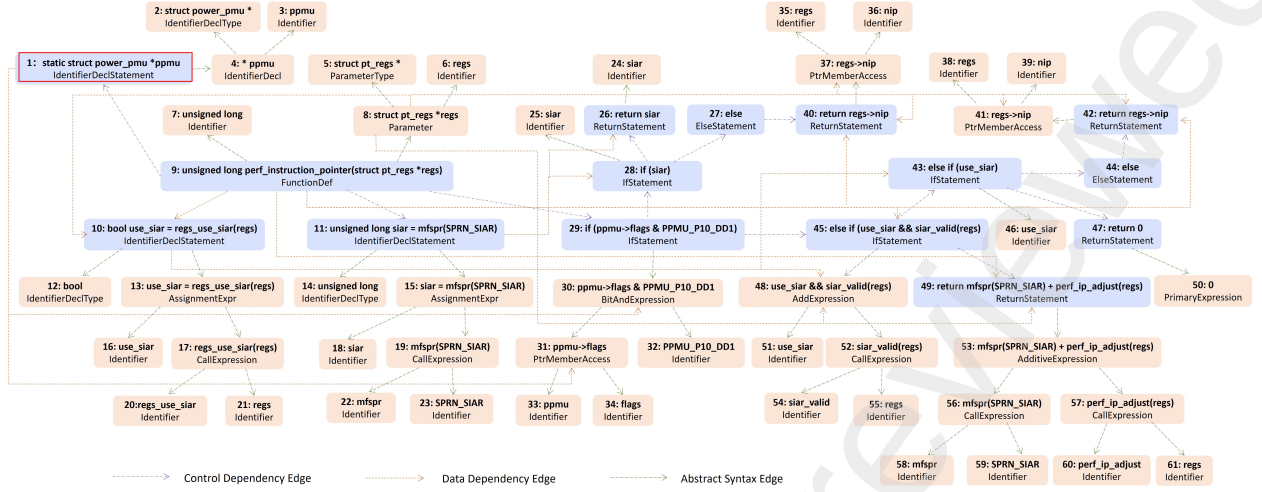


Fig. 3. Code Property Graph for the instance in Fig. 2a

3.2.2. Property Graph-based Program Slicing

After extracting SEPs, we perform program slicing by traversing the CPG to generate the PrVC. Since a SEP can affect subsequent nodes, and may also be influenced by preceding nodes, we construct the node set of the PrVC by performing both forward and backward slicing. In forward slicing, we traverse all forward paths starting from the SEP and search for nodes that are reachable from the SEP, forming the forward node set. In the backward slicing, we traverse all backward paths starting from the SEP to generate the backward node set. We combine the forward and backward node sets to form the complete reachable node set.

Algorithm 1 outlines the process of generating PrVCs. As shown, *ForwardCPGSlice* and *BackwardCPGSlice* are both iterative functions used to determine the forward and backward node sets, respectively. The pseudocode for *BackwardCPGSlice* is similar to that of *ForwardCPGSlice*, with the main difference being that *ForwardCPGSlice* searches for predecessor nodes while *BackwardCPGSlice* searches for successor nodes. Consequently, we have omitted the pseudocode for *BackwardCPGSlice* in Algorithm 1. The complexity of Algorithm 1 is $O(n^2)$. It is worth noting that multiple SEPs can be extracted from each Code Property Graph, and each SEP can generate its own PrVC. Therefore, the number of SEPs is equal to the number of PrVCs.

For the Code Property Graph in Fig. 3, we apply program slicing using Algorithm 1. We traverse the CPG starting from Node 1. Initially, the reachable node set only contains Node 1. During the first iteration, we discover that Node 1 is control dependent on Node 9 with an Edge $9 \rightarrow 1$, and Node 4 is syntax dependent on Node 1 with an edge $1 \rightarrow 4$. Node 30 and Node 31 are data dependent on Node 1 with Edges $1 \rightarrow 30$ and $1 \rightarrow 31$. Consequently, we update the reachable node set by including Node 4, 9, 30 and 31. In the subsequent iterations, we systematically search for nodes that have node dependencies with Node 4, 9, 30 and 31, continually updating the reachable node set. Ultimately, this process allows us to generate a PrVC, as illustrated in Fig. 4. Notably, the number of nodes in the PrVC is significantly lower than that in the CPG. When handling large functions with many lines of code, our method effectively filters out irrelevant nodes through graph-level slicing. Furthermore, to preserve the natural sequential order of the source code, we connect neighboring leaf nodes in the PrVC based on the token order in the source code, forming code sequence edges in the PrVC, represented by blue dashed lines in Fig. 4.

Then we label each PrVC as follows: if at least one node in the PrVC corresponds to a vulnerable statement in the source code, the PrVC is labeled as '1' (i.e., vulnerable); otherwise, it is labeled as '0' (i.e., non-vulnerable). In Fig. 4, the PrVC is labeled as '1' because it contains at least one node that matches the vulnerable statement 'if (ppmu $\rightarrow$ flags & PPMU_P10_DD1)'.

Algorithm 1: Generating PrVCs

Input: Code Property Graph G_i ; A set of SEPs d_i

Output: The set of PrVCs GS

```

1  $GS = \emptyset$ 
2 for each SEP  $n_{i,j} \in d_i$  do
3    $reach_{i,j} = \{n_{i,j}\}$ 
4   for each node  $n_{i,k} \in reach_{i,j}$  do
5     Traverse the node dependencies in  $G_i$  to obtain the predecessor node set  $DP_i$ 
6     for each node  $n_{i,l} \in DP_i$  do
7        $ForwardCPGSllice(n_{i,l}, reach_{i,j})$ 
8     end
9   end
10  for each node  $n_{i,k} \in reach_{i,j}$  do
11    Traverse the node dependencies in  $G_i$  to obtain the successor node set  $DS_i$ 
12    for each node  $n_{i,m} \in DS_i$  do
13       $BackwardCPGSllice(n_{i,m}, reach_{i,j})$ 
14    end
15  end
16  Connect nodes in  $reach_{i,j}$  based on their edges to create the PrVC  $G_i^j$ 
17   $GS = GS \cup G_i^j$ 
18 end
19 Function  $ForwardCPGSllice(n_{i,l}, reach_{i,j})$ 
20   if  $n_{i,l} \notin reach_{i,j}$  then
21      $reach_{i,j} = reach_{i,j} \cup n_{i,l}$ 
22     Traverse the node dependencies in  $G_i$  to obtain the predecessor node set  $C_i$ 
23     for each node  $n_{i,p} \in C_i$  do
24        $ForwardCPGSllice(n_{i,p}, reach_{i,j})$ 
25     end
26   end
  
```

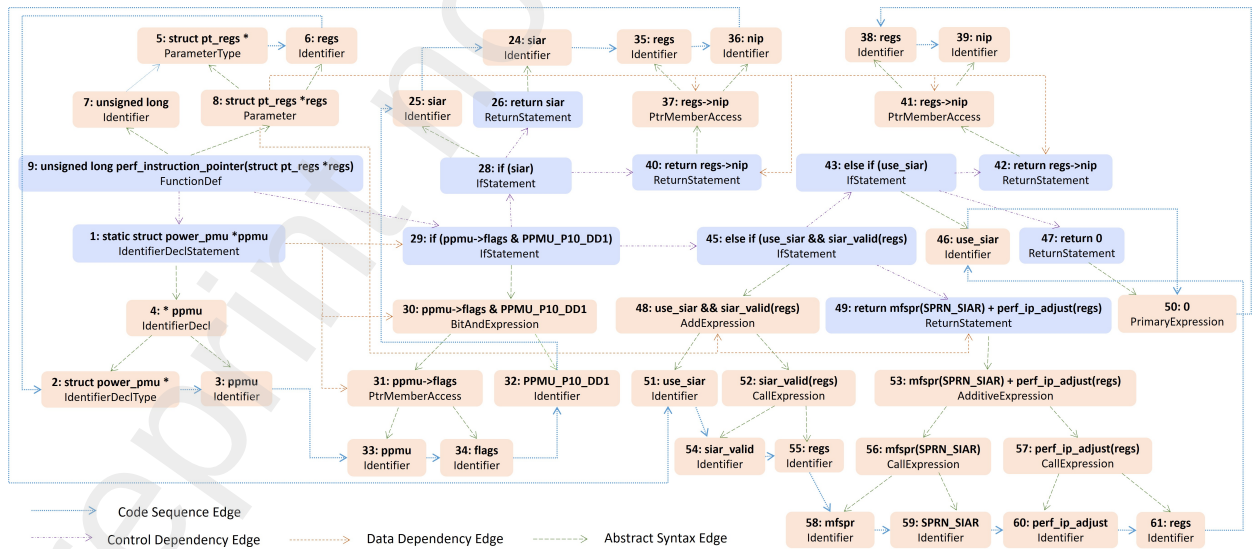


Fig. 4. A PrVC extracted from the Code Property Graph in Fig. 3

3.3. Fine-grained Vulnerability Detection

3.3.1. Graph Learning via Hierarchical Attention Graph Convolutional Network

To learn the embedding representations of PrVCs, we develop a Hierarchical Attention Graph Convolutional Network (HierGCN) that encodes the PrVCs into vector representations. HierGCN introduces hierarchical attention in Relational Graph Convolutional Networks, incorporating both local attention and global attention. This hierarchical attention mechanism enables us to fuse features at different levels based on their importance. As shown in Fig. 5, the HierGCN framework consists of three stages: (i) local attention for node embedding, (ii) node update via RGCN layer, and (iii) global attention for graph embedding. In the first stage, the vectors of all nodes in each PrVC are initialized. In the second and third stages, the node vectors are updated and fused to produce the comprehensive graph embedding vectors for each PrVC.

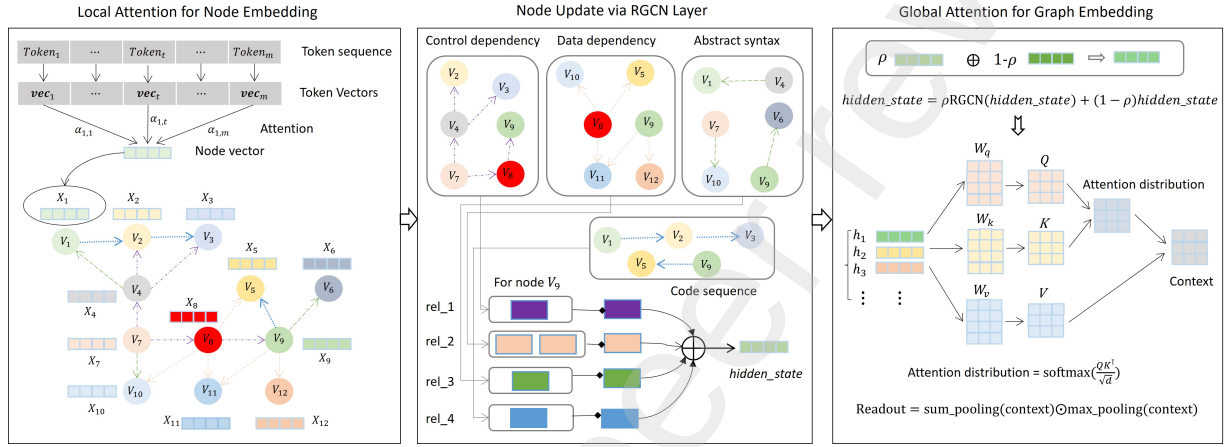


Fig. 5. The graph learning process in our graph learning framework (HierGCN)

Local Attention for Node Embedding. The pre-trained model GraphCodeBERT [34] has demonstrated notable improvements in various programming language (PL) downstream tasks, including code search and code translation. To capitalize on the advantages of pre-trained PL models, we use GraphCodeBERT to initialize the node vectors of the PrVCs.

Given a function f_i with its CPG $G_i = (V_i, E_i)$, we generate a PrVC $G'_i = (V'_i, E'_i)$ as described in the previous subsection, where $V'_i = \{v_{i,1}, \dots, v_{i,w}\}$ represents an ordered set of nodes. For each node $v_{i,j}$ ($1 \leq j \leq w$), its code is transformed into a token sequence, and GraphCodeBERT encodes each token into a 768-dimensional vector. Assuming that node $v_{i,j}$ is composed of m tokens, its features are represented as $\text{vec}_{i,j} = \{\text{vec}_{i,j,1}, \dots, \text{vec}_{i,j,m}\}$. Not all tokens contribute equally to a node's semantics. To capture the significance of key tokens, we employ a local attention, which emphasizes the semantics of more important tokens. This mechanism aggregates the representations of all tokens for each node to derive the node vector. The attention distribution and node vector are computed as follows:

$$\alpha_{i,j,t} = \text{softmax}(\omega_2^T \tanh(\omega_1 \text{vec}_{i,j,t} + b_1)) \quad (1)$$

$$X'_{i,j} = \sum_t \alpha_{i,j,t} \text{vec}_{i,j,t} \quad (2)$$

where ω_1 , ω_2 and b_1 are learnable weights and bias, respectively. $X'_{i,j}$ represents the fused embedding vector of node $v_{i,j}$. Thus, we have obtained the initial node vector matrix $X'_i = \{X'_{i,1}, \dots, X'_{i,w}\}$.

Node Update via RGCN Layer. After initializing the node vectors for each PrVC, we construct a node update layer. Most existing GNN-based vulnerability detection approaches rely on Graph Convolutional Network (GCN) or Gated Graph Neural Network (GGNN) to update node vectors, but these methods often fail to account for the various dependencies between nodes. In this study, we model the PrVC as a directed multi-relational graph, representing it through triplets that combine nodes and edges. To better capture the relationships within the graph, we propose a novel node update method based on the Relational Graph Convolutional Network (RGCN) [35]. As illustrated in Fig. 5,

this approach aggregates the features of neighboring nodes to update the central node's features, while explicitly considering different edge types.

Given the PrVC $G'_i = (V'_i, E'_i)$, we convert it into a directed multi-relational graph $G'_i = (V'_i, E'_i, R'_i)$ with nodes $v_i \in V'_i$ and labeled edges $(v_i, r, v_j) \in E'_i$, where $r \in R'_i$ represents a relation type. The RGCN model can be expressed as follows:

$$\mathbf{h}_{i,j}^{(\tau+1)} = \varphi \left(\sum_{r \in R'_i} \sum_{k \in N_{i,j}^r} \frac{1}{c_{i,j,r}} \mathbf{W}_r^{(\tau)} \mathbf{h}_{i,k}^{(\tau)} + \mathbf{W}_0^{(\tau)} \mathbf{h}_{i,j}^{(\tau)} \right) \quad (3)$$

where $N_{i,j}^r$ represents the set of neighboring nodes of node j under relation $r \in R'_i$. The normalization constant $c_{i,j,r}$ is specific to the problem and can either be pre-learned or pre-determined. In this paper, we set $c_{i,j,r} = |N_{i,j}^r|$. The layer-specific learnable weight matrices $\mathbf{W}_r^{(\tau)}$ and $\mathbf{W}_0^{(\tau)}$ aggregate information from neighboring nodes based on their importance to the central node. The output of the τ -th layer for node j is denoted as $\mathbf{h}_{i,j}^{(\tau)}$, and the function $\varphi(\cdot)$ applies the ReLU activation. The initial node features are represented as $\mathbf{h}_{i,j}^{(0)} = \mathbf{X}'_{i,j}$. This propagation process updates the central node by normalizing and accumulating the transformed features of its neighboring nodes.

To enhance RGCN's capacity for handling large vulnerability datasets, a natural approach is to increase the number of RGCN layers. However, directly stacking multiple RGCN layers can result in noise propagation from the exponentially growing extended neighborhood, which impedes performance optimization. Inspired by [36], we incorporate residual connections to address this challenge. Residual connections allow gradients to pass directly through the layer, effectively mitigating vanishing or exploding gradients problems. Accordingly, we redefine the node feature matrix update process as follows:

$$\mathbf{h}_{i,j}^{(\tau+1)} = \rho \text{RGCN}(\mathbf{h}_{i,j}^{(\tau)}) + (1 - \rho) \mathbf{h}_{i,j}^{(\tau)} \quad (4)$$

where $\rho \in [0, 1]$ is an adjustment parameter. The state vector for node j at time step τ is denoted as $\mathbf{h}_{i,j}^{(\tau)}$. Assuming a total of T update steps, the final state vector for node j is represented as $\mathbf{h}_{i,j}^{(T)}$.

Global Attention for Graph Embedding. To obtain the overall graph representation, we aggregate node representations using global attention. Previous approaches often involve directly summing or averaging node features, but this method overlooks the varying influence of individual nodes on the graph. In this paper, we apply global attention to assign different weights to nodes based on their importance. Additionally, we combine sum pooling and max pooling to integrate the weighted vectors of all nodes, producing a graph-level embedding representation. Consequently, the overall graph embedding of PrVC G'_i is computed as follows:

$$\beta_{i,j} = \text{softmax} \left(\frac{(\mathbf{W}_q \mathbf{h}_{i,j}^{(T)}) (\mathbf{W}_k \mathbf{h}_{i,j}^{(T)})^\top}{\sqrt{d}} \right) \quad (5)$$

$$\mathbf{h}_{G'_i} = \text{sumpool}\{\beta_{i,j} \mathbf{W}_v \mathbf{h}_{i,j}^{(T)} | 1 \leq j \leq w\} \odot \text{maxpool}\{\beta_{i,j} \mathbf{W}_v \mathbf{h}_{i,j}^{(T)} | 1 \leq j \leq w\} \quad (6)$$

where $\mathbf{h}_{i,j}^{(T)}$ represents the node vector for node j in G'_i , updated at time step T . The terms $\mathbf{W}_q \mathbf{h}_{i,j}^{(T)}$, $\mathbf{W}_k \mathbf{h}_{i,j}^{(T)}$, and $\mathbf{W}_v \mathbf{h}_{i,j}^{(T)}$ correspond to the query, key and value vectors in the global attention, respectively. This global attention determines which nodes are relevant to the vulnerability.

Global attention captures overarching information by evaluating the degree of association between each node and other nodes. In contrast, local attention focuses on finer-grained features within the local context of the vulnerable code, particularly the dependency relationships between different tokens on nodes. By combining both local attention and global attention in our hierarchical attention mechanism, the model can simultaneously extract and integrate token-level and node-level features. This leads to a more accurate and comprehensive code representation, thereby improving the performance and accuracy of vulnerability detection models.

3.3.2. Vulnerability Prediction

Given a PrVC $G'_i = (V'_i, E'_i)$ of function f_i , we input the graph embedding $\mathbf{h}_{G'_i}$ of G'_i into a two-layer MLP, followed by sigmoid function, to predict whether the given PrVC is vulnerable. The choice of MLP is motivated by its ability

to achieve high recognition accuracy while offering fast prediction speeds. The prediction process of the MLP is as follows:

$$\hat{y}'_i = \text{sigmoid}(\text{MLP}(\mathbf{h}_{G'_i})) \quad (7)$$

Furthermore, we define a sample as $\{(G'_i, y'_i) | G'_i \in GS, y'_i \in Y\}$, where $Y = \{0, 1\}^N$ denotes the label set, and N represents the total number of PrVCs. We train and optimize GraphFVD by minimizing the following loss function:

$$\min \sum_{i=1}^N L(\hat{y}'_i, y'_i) + \lambda \|\theta\|_2^2 \quad (8)$$

where $L(\cdot)$ represents the cross-entropy loss function, θ denotes the model parameters and λ is an adjustable weight of the $L2$ regularization term.

3.4. A Theoretical Comparison of Code Representation in GraphFVD and Other Slice-based Methods

Since GraphFVD, VulDeePecker [20], SySeVR [1], and MVD [21] are all slice-based methods, we highlight the key distinctions between our approach, GraphFVD, and these three methods. VulDeePecker and SySeVR focus on extracting sequence-level program slices from PDG. However, sequence-level program slices tend to ignore critical structural features of the program, making it difficult to capture the essential characteristics of vulnerable code. MVD, extracts graph-level program slices from PDG. While these graph-based slices retain the program's structural information, the PDG still struggles to represent the natural sequential order and abstract syntax details of code statements. Therefore, these graph-based slices still have limitations in detecting vulnerabilities.

Unlike the above three approaches, our GraphFVD extracts property graph-based slices (i.e., PrVCs) from CPG. To select SEPs, GraphFVD considers the following code characteristics: function calls, sensitive variables, arithmetic operators, and if-condition statements. The use of SEPs improves the efficiency of generating slices that are closely related to vulnerabilities. Additionally, GraphFVD simultaneously accounts for control flow, data flow, syntax dependency, and natural sequential order of the source code when constructing PrVCs. This ensures that PrVCs capture both the fundamental structural features of the CPG and the natural sequence of the source code.

4. Experimental Setup

4.1. Research Questions

To evaluate the effectiveness of our proposed approach GraphFVD, our experiments focus on answering the following Research Questions (RQs):

- RQ1: How effective is GraphFVD compared to the state-of-the-art vulnerability detection approaches?
- RQ2: Is our PrVC more effective for vulnerability detection than other code representations?
- RQ3: How effective is our proposed graph learning framework HierGCN in detecting vulnerabilities?
- RQ4: What is the impact of each part of HierGCN on its overall detection performance?

Implementation. In order to answer these questions, we implement GraphFVD in Python using Pytorch³. The experiments are conducted on a Linux server equipped with NVIDIA GeForce RTX 4090 GPU and Intel(R) Core(TM) i9-13900K CPU running at 4.60 GHz. We utilize the open-source tool Joern⁴ to parse the source code and generate the Code Property Graph (CPG). The main hyperparameters used to train GraphFVD are described as follows: the Adam optimizer with the learning rate of $5e-4$ is exploited to train the model up to 200 epochs; the batch size is set to 128; the hidden size for HierGCN is 128; the number of layers is 2; the number of nodes in each PrVC is set to 400.

Evaluation Metrics. In this paper, we follow [1] and use the following metrics to evaluate the effectiveness of our model: accuracy (ACC), precision (P), F1 score (F1), AUC, false positive rate (FPR), and false negative rate (FNR).

³<https://pytorch.org/>

⁴<https://joern.io/>

4.2. Datasets

Training deep learning models for vulnerability detection requires large datasets. However, most existing methods rely on synthetic datasets, such as SARD [37], which lack the diversity found in real-world programs. Although datasets like those used in Devign [4] and ReGVD [25] contain real-world functions, they are unsuitable for slice-based vulnerability detection as they only provide function-level vulnerability labels. To evaluate GraphFVD’s performance in detecting C/C++ vulnerabilities at a slice level, we constructed our datasets from two sources: (1) the well-known real-world dataset NVD [38], widely used in previous detection studies [1, 2, 20, 21]; and (2) a new dataset named GITA, comprising four real-world open-source projects—OpenSSL, Libav, FFmpeg, and Linux Kernel—collected from the GitHub repository.

Training deep learning-based detectors for fine-grained vulnerability detection requires large-scale datasets. Therefore, we followed the methodology outlined in [39] to construct the GITA dataset using four open-source projects: OpenSSL, Libav, FFmpeg, and Linux Kernel. These projects have diverse functions, and have been widely used in previous studies. The data collection process is as follows: for the Linux Kernel project, we first identified each vulnerability’s Github link in the CVE security vulnerability database based on its CVE ID. Using these links, we retrieved both the vulnerable code and the corresponding patch code. For the other three projects (OpenSSL, Libav, FFmpeg), we downloaded the source code associated with vulnerabilities by filtering commit messages on Github. Specifically, all commit messages in each project were examined and labeled as either vulnerability fixing commits or non-vulnerability fixing commits. From the labeled commits, we extracted functions related to vulnerabilities.

We present the statistics of our datasets in Table 1. Our datasets consist of a total of 32,466 C/C++ functions, out of which 10,666 are vulnerable. Column 3 shows the range of project versions affected by vulnerabilities in our datasets. Column 4 represents the number of vulnerable functions, while column 5 indicates the total number of functions. In terms of PrVCs, column 6 displays the number of vulnerable PrVCs, and column 7 represents the total number of PrVCs derived from the functions. Table 1 reveals that we extracted a total of 412,066 PrVCs, out of which 90,246 are vulnerable. To evaluate our model, we randomly shuffled each dataset and allocated 80% of the PrVCs for training, reserving the remaining 20% for validation and testing.

Table 1: Statistics on datasets.

Dataset	Project	#Version	#Vul. Fs	#Fs	#Vul. PrVCs	#PrVCs
NVD [38]	-	-	937	2,011	9,683	36,571
GITA (Ours)	OpenSSL	0.9.6-3.0.7	2,009	4,537	13,349	46,500
	Libav	0.6-11.5	1,666	7,990	13,894	97,960
	FFmpeg	0.3-5.1	4,876	14,900	44,533	194,734
	Linux Kernel	2.6-5.17	1,178	3,028	8,787	36,301

According to our statistics, approximately 12 SEPs can be extracted from each CPG, resulting in the generation of 12 PrVCs. Besides, the average number of nodes in CPG is 307, while the number of nodes in PrVC is approximately half that of the CPG. The code representation of PrVC contributes to enhancing the efficiency of GNN-based vulnerability detection models.

Additionally, we examined the distribution of vulnerability types in the Linux Kernel project within the GITA dataset. In total, the Linux Kernel project contains over 56 types of known vulnerabilities, each uniquely identified by a Common Weakness Enumeration identifier (CWE ID). The distribution of these vulnerability types in the Linux Kernel project is shown in Fig. 6, with CWE-119 (improper restriction of operations within the bounds of a memory buffer) accounting for the largest proportion at 11.76%.

4.3. Baseline Methods

We evaluate GraphFVD against two groups of baseline methods. The first group consist of 10 previous vulnerability detection methods. Next, we provide a brief introduction to these methods.

SySeVR [1]: SySeVR analyzes the syntax features of vulnerabilities based on the node attributes of AST. It extracts sequence-level slices from PDG and uses Bidirectional Gated Recurrent Units (BGRU) to detect vulnerabilities.

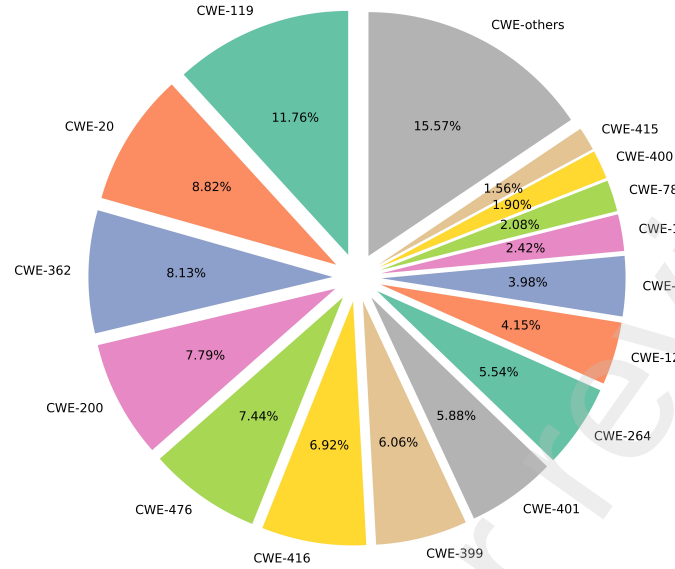


Fig. 6. Distribution of vulnerability types in the Linux Kernel Project

VulDeeLocator [2]: This approach leverages program intermediate code to capture semantic information, generate program slices, and employs BGRU to detect and locate vulnerabilities.

CNN [3]: This approach parses the source code into a token sequence and employs a Convolutional Neural Network (CNN) to learn features and detect vulnerabilities.

Devign [4]: Devign transforms a function into a CPG, and uses a Gated Graph Recurrent Network (GGRN) to obtain the graph embedding of the CPG, enabling vulnerability prediction.

Flawfinder [14]: Flawfinder is a traditional rule-based vulnerability detection tool. It exploits expert-crafted rules to analyze C/C++ source code and identify possible vulnerabilities.

VulDeePecker [20]: VulDeePecker is a slice-based vulnerability detection method that extracts sequence-level program slices (i.e., code gadgets) from PDG. It is specifically designed to detect vulnerabilities associated with library/API function calls.

MVD [21]: MVD extracts graph-level program slices from the PDG and utilizes a flow-sensitive Graph Neural Network to learn slice representations and detect vulnerabilities.

ReGVD [25]: ReGVD considers a function as a token sequence and then construct a graph representation.

VulChecker [24]: VulChecker is a statement-level detection system that constructs a novel code graph, ePDG, from the LLVM intermediate representation (IR) and employs the Structure2Vec (S2V) model to learn graph embeddings.

GRACE [33]: GRACE is an LLM-based detection approach. It employs a demonstration retrieval technique to identify code examples highly relevant to the target code, offering improved demonstrations for in-context learning.

The second group comprises several Large Language Models (LLMs) for code. LLMs have gained increasing popularity for various downstream software tasks due to their robust generalization capabilities. To compare GraphFVD with LLMs, we evaluate three well-known code LLMs for vulnerability detection: ChatGPT-3.5-Turbo-1106, GPT-4-0613 and DeepSeek-Coder-V2. Among them, ChatGPT-3.5-Turbo-1106⁵ and GPT-4-0613⁶, developed by OpenAI, are based on the GPT-3.5 and GPT-4 architectures, respectively. Both models excel in handling tasks involving both natural language and programming languages. DeepSeek-Coder-V2⁷ is a series of code models, each trained from

⁵<https://openai.com/blog/chatgpt>

⁶<https://openai.com/research/gpt-4>

⁷<https://chat.deepseek.com>

scratch on 2 trillion tokens. Evaluation on programming-related benchmarks demonstrates that DeepSeek-Coder-V2 outperforms most existing open-source and closed-source LLMs [40]. All three models use the following prompt as input: “{code} In the above code snippet, check for potential security vulnerabilities and output either ‘Vulnerable’ or ‘Non-vulnerable’ ”. Here, the {code} represents the entire input function for vulnerability detection.

5. Experimental Results

5.1. Experiments for Answering RQ1

To demonstrate the effectiveness of GraphFVD, we compare it against state-of-the-art vulnerability detection methods and LLMs across two datasets, as presented in Table 2 and Table 3. Specifically, for the approaches Flawfinder [14], CNN [3], VulDeePecker [20], ReGVD [25], and SySeVR [1], we utilize the replication packages provided by the authors and re-implement them on our datasets. Since the source code for MVD [21] and VulDeeLocator [2] is not publicly available, we reference their results directly from the original papers. This is feasible because these methods are evaluated on the same dataset, i.e., NVD.

As shown in Table 2 and Table 3, the optimal results are highlighted in bold, while the sub-optimal results are underlined. We observe that GraphFVD outperforms all baseline methods across nearly all metrics on both datasets. Flawfinder, a rule-based vulnerability detection approach, suffers from high false negative and false positive rates. In practice, vulnerabilities encompass a wide range of types, with significant variations in their manifestations across different code files. Thus, expert-crafted detection rules struggle to effectively address the majority of vulnerabilities.

Table 2: Comparison of GraphFVD and baseline approaches on the NVD dataset (metrics unit: %).

Type	Approach	ACC	F1	P	FNR	FPR
Previous Method	Flawfinder	59.27	59.91	55.33	34.69	46.00
	CNN	68.49	68.80	74.87	36.82	25.95
	VulDeePecker	71.51	62.77	52.35	21.68	31.54
	MVD [21]	74.00	65.20	61.50	30.63	23.50
	Devign [4]	74.33	73.07	-	-	-
	ReGVD	78.11	78.43	74.07	<u>17.71</u>	27.10
	VulDeeLocator [2]	83.90	80.00	81.80	21.70	12.10
	SySeVR	<u>87.14</u>	78.72	81.99	20.81	8.52
LLM	ChatGPT-3.5-Turbo-1106	53.23	58.41	49.25	29.35	62.16
	DeepSeek-Coder-V2	71.14	68.13	59.05	19.48	34.68
	GPT-4-0613	85.43	<u>81.76</u>	90.28	26.44	<u>7.02</u>
Ours	GraphFVD	91.11	82.92	<u>82.27</u>	16.53	6.30

Owing to the automatic feature extraction capability of deep learning models, methods such as CNN, ReGVD, VulDeePecker, SySeVR and GraphFVD outperform Flawfinder. However, both CNN and ReGVD focus solely on function-level code representation, neglecting fine-grained program dependency semantics, which results in inferior performance compared to GraphFVD. As shown in Table 2, Devign achieves notable results on the NVD dataset, with an accuracy of 74.33% and an F1 score of 73.07%. However, our GraphFVD surpasses Devign, improving accuracy and F1 score by 16.78% and 9.85%, respectively. This demonstrates that while leveraging GNNs to extract semantic features of vulnerable code from CPG significantly enhances Devign’s performance, the inclusion of redundant nodes unrelated to vulnerabilities in the graph still hinders its ability to effectively detect vulnerabilities.

VulDeePecker, SySeVR, and VulDeeLocator are all slice-based detection methods. Among them, VulDeePecker performs significantly worse than SySeVR and VulDeeLocator because it only considers vulnerability characteristics related to API function calls, neglecting control dependencies in the source code. Additionally, we observe that GraphFVD reduces the false negative rate and false positive rate by 5.07% and 3.59%, respectively, compared to SySeVR on the GITA dataset. This improvement is primarily due to the effectiveness of GraphFVD’s graph-level slices, i.e., PrVCs, which outperform the sequence-level slices generated by SySeVR. Moreover, GraphFVD leverages

Table 3: Comparison of GraphFVD and baseline approaches on the GITA dataset (metrics unit: %).

Type	Approach	ACC	F1	P	FNR	FPR
Previous Method	Flawfinder	53.57	41.80	41.42	57.81	41.32
	CNN	58.93	50.76	48.32	46.53	40.77
	GRACE [33]	60.13	66.38	54.58	15.32	61.19
	ReGVD	69.23	58.33	63.32	45.98	20.74
	Devign [4]	72.26	73.26	-	-	-
	VulDeePecker	83.41	63.85	56.89	27.30	16.53
	SySeVR	<u>87.23</u>	<u>75.92</u>	<u>77.45</u>	25.56	<u>8.20</u>
LLM	ChatGPT-3.5-Turbo-1106	52.86	52.86	45.80	38.54	54.14
	DeepSeek-Coder-V2	63.87	60.39	56.82	35.57	36.54
	GPT-4-0613	72.25	67.02	71.11	36.63	20.63
Ours	GraphFVD	92.05	80.83	82.18	<u>20.49</u>	4.61

HierGCN to learn graph embeddings of PrVCs, overcoming the limitations of sequence-based models like BGRU in capturing comprehensive vulnerability semantics. Furthermore, GraphFVD outperforms VulDeeLocator across all metrics. This indicates that while VulDeeLocator designs program slices by converting source code into intermediate code, the sequence-level slices derived from intermediate code remain insufficient for effective vulnerability detection. MVD is the most similar to our approach, as both are vulnerability detection methods based on graph-level slices. As shown in Table 2, GraphFVD demonstrates improvements in accuracy, F1 score, and precision over MVD by 17.11%, 17.72%, and 20.77%, respectively. This result can be explained that our PrVC extracted from Code Property Graph is more effective in identifying vulnerability features than the property graph-based slices extracted from Program Dependency Graph by MVD. In addition, our proposed graph learning framework HierGCN also contributes to the high performance of GraphFVD.

Moreover, we compared GraphFVD with the state-of-the-art LLM-based detection method, GRACE. The experimental result shows that GraphFVD outperforms GRACE across all metrics, with a notable increase in the F1 score by 14.45%. We also observe that GraphFVD significantly outperforms three LLMs. Compared to the best LLM, GPT-4, GraphFVD improves accuracy, F1 score, and precision by 19.80%, 13.81%, and 11.07%, respectively, on the GITA dataset. These findings highlight that LLMs, such as ChatGPT-3.5, GPT-4 and DeepSeek-Coder, struggle to effectively detect real-world software vulnerabilities because they cannot update their parameters to fully leverage the training data. Another reason is that LLMs are typically pre-trained on text and code generation datasets, which lack the specific-domain knowledge for detecting vulnerabilities.

Additionally, to compare our GraphFVD with VulChecker, we evaluate GraphFVD using the AUC metric on the Juliet dataset [24]. This dataset encompasses five vulnerability types: integer overflow (CWE-190), stack overflow (CWE-121), heap overflow (CWE-122), double free (CWE-415), and use-after-free (CWE-416). As shown in Table 4, GraphFVD demonstrates overall competitive performance with VulChecker across these vulnerability types. This demonstrates that both our PrVC, extracted from the source code, and VulChecker’s ePDG, derived from LLVM IR, are effective in capturing fine-grained code semantics. Notably, GraphFVD achieves a 10.78% higher AUC than VulChecker on CWE-122. This improvement is likely because heap memory allocation and release often involve numerous API and library function calls. By generating program slices that analyze these relevant functions, GraphFVD more effectively learns vulnerability features, leading to improved detection of this vulnerability type.

Table 4: Comparison of VulChecker and GraphFVD on the Juliet dataset.

Method	CWE-190	CWE-121	CWE-122	CWE-415	CWE-416
VulChecker [24]	97.00	85.40	79.00	100.00	90.90
GraphFVD	92.82	89.93	89.78	92.07	94.96

5.2. Experiments for Answering RQ2

In this experiment, we compare the effectiveness of different code representations in vulnerability detection, considering two key factors: graph structure (PDG and CPG) and detection granularity (function, sequence-level slice and graph-level slice). To compare our code representation PrVC with the sequence-level slice, we exploit GraphCodeBERT to embed tokens and employ TextCNN to learn the embeddings of the sequence-level slice for vulnerability detection. TextCNN is a powerful model for long text classification. For function and property graph-based slice representations, we initialize node embeddings using GraphCodeBERT and then apply GCN to learn graph embeddings and detect vulnerabilities. This experiment is conducted on the GITA dataset.

As shown in Table 5, CPG+Function represents that the direct transformation of the source code’s functions into a CPG. This type of code representation has been widely used in previous studies [4, 11]. PDG+Sequence_slice refers to the program slicing technique used in VulDeePecker [20] and [1] SySeVR to generate sequence-level slices from PDG. PDG+Graph_slice generates graph-level slices from PDG, similar to the code representations proposed in [21, 24]. Both CPG+Sequence_slice and CPG+Graph_slice are the code representations introduced in this paper. Notably, CPG+Graph_slice corresponds to PrVC, which involves generating property graph-based slices from CPG.

Table 5: The performance comparison of different code representations.

Granularity	Code Representation	ACC	F1	P	FNR	FPR
Function-level	PDG+Function	61.59	49.50	45.86	46.65	35.34
	CPG+Function	65.42	52.74	46.72	39.57	34.68
Slice-level	PDG+Sequence_slice	82.96	68.44	68.38	31.38	11.79
	CPG+Sequence_slice	<u>88.53</u>	<u>74.69</u>	<u>74.79</u>	<u>25.48</u>	<u>7.49</u>
	PDG+Graph_slice	87.32	73.31	72.17	25.57	9.09
	CPG+Graph_slice (PrVC)	90.83	78.67	77.84	20.43	6.33

The experimental results are shown in Table 5, with the optimal results highlighted in bold and the sub-optimal results underlined. Overall, PrVC outperforms the other five code representations across all evaluation metrics. Within the group with the same granularity, we observe that CPG is more effective than PDG. This can be explained that CPG captures richer semantic information, such as syntax dependency and execution orders of statements, thereby covering more types of vulnerabilities. Moreover, we find that when using the same graph structure, code representations with different granularities yield notable performance differences in vulnerability detection. Specifically, PrVC significantly outperforms the function-level representation (CPG+Function) with improvements of 25.41% in accuracy, 25.93% in F1 score, respectively. Additionally, PrVC consistently outperforms the sequence-level slice extracted from the CPG (CPG+Sequence_slice). These findings suggest that our proposed property graph-based slices (i.e., PrVCs) are more effective in discovering vulnerabilities compared to function-level representations and sequence-level slices.

5.3. Experiments for Answering RQ3

After comparing the effectiveness of our PrVC with other code representations, we proceed to evaluate the effectiveness of our proposed graph learning framework. In this experiment, we first replace GraphCodeBERT with two alternative state-of-the-art pre-trained models, i.e., BERT [41] and CodeBERT [42], to assess the performance of different node embedding models.

Fig. 7 presents the experimental results of three different node embedding models across four projects in the GITA dataset. It is observed that GraphCodeBERT slightly outperforms CodeBERT. This can be attributed to the fact that GraphCodeBERT retains the structural information of the AST during the code embedding process. Additionally, BERT exhibits the lowest performance in terms of almost all metrics. The fundamental reason for this gap is that BERT is trained on natural language data and may not be suitable for embedding source code of programs.

Then we proceed to compare HierGCN with GCN and RGCN to analyze the effectiveness of HierGCN in vulnerability detection. Table 6 presents a performance comparison among GCN, RGCN, and HierGCN across different projects. It is evident that RGCN outperforms GCN. For instance, in the Linux Kernel project, RGCN exhibits improvements over GCN, with an increase of 0.64% in accuracy, 1.23% in F1 score, and 1.28% in precision. This can be

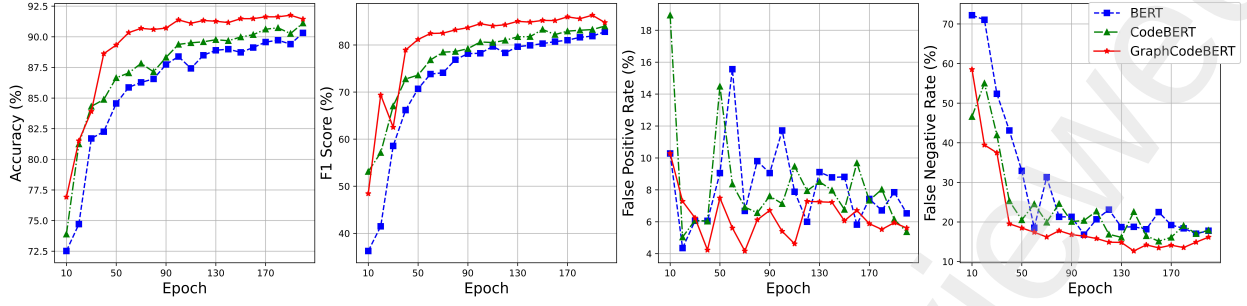


Fig. 7. Comparison of different node embedding models

explained by the fact that RGCN employs different parameter matrices for neighboring nodes corresponding to various types of edges to model the heterogeneous relationships of edges, which is more effective in learning the embedding representation of PrVCs. Compared to GCN and RGCN, our proposed HierGCN achieved optimal performance on all projects. This significant advantage is primarily due to the hierarchical attention mechanism in HierGCN, which assigns different weights to tokens or nodes based on their importance. This enables more effective capture of code vulnerability semantics, leading to improved vulnerability detection.

Table 6: Comparative results among GCN, RGCN and HierGCN across different projects.

Project	Model	ACC	F1	P	FNR	FPR
OpenSSL	GCN	91.44	84.76	85.61	16.15	5.61
	RGCN	91.33	84.86	86.18	16.51	5.51
	HierGCN	91.81	85.50	85.86	14.94	5.58
Libav	GCN	92.91	74.20	74.31	25.98	4.10
	RGCN	93.53	76.84	78.92	25.20	3.36
	HierGCN	94.00	77.93	79.36	23.53	3.21
FFmpeg	GCN	90.41	78.76	78.75	21.24	6.21
	RGCN	90.59	78.94	79.76	21.88	5.79
	HierGCN	91.17	80.62	82.13	20.85	5.21
Linux Kernel	GCN	88.54	76.97	72.70	18.35	9.41
	RGCN	89.18	78.20	73.98	17.18	8.95
	HierGCN	91.35	81.89	80.32	16.59	6.29

5.4. Experiments for Answering RQ4

In this experiment, we conduct an ablation analysis to investigate the effects of different components of GraphFVD, including the impacts of local attention, global attention, and the presence or absence of residual connections. As shown in Table 7, GraphFVD-RC refers to the model where residual connections are removed, and multiple RGCN layers are directly stacked for training the graph learning model. GraphFVD-RC-local_attention indicates that both residual connections and local attention are removed from GraphFVD. GraphFVD-RC-local_attention-global_attention means that all three components—local attention, global attention, and residual connections—are removed from GraphFVD.

The experimental results are presented in Table 7. As we progressively remove essential components of GraphFVD, the overall experimental results on various metrics gradually decrease. Initially, when residual connections are not used to connect different RGCN layers (i.e., GraphFVD-RC), the model’s evaluation results on the GITA dataset show a slight decrease. This demonstrates that residual connections are beneficial for GraphFVD in more effectively learning vulnerability features, consequently improving the performance of vulnerability detection.

Table 7: Ablation study on GraphFVD.

Model	ACC	F1	P	FNR	FPR
GraphFVD	92.08	81.49	81.92	18.98	5.07
GraphFVD-RC	91.71	80.78	81.46	19.93	5.20
GraphFVD-RC-local_attention	91.06	79.89	79.89	19.96	6.11
GraphFVD-RC-local_attention-global_attention	90.89	79.17	79.91	21.54	5.82

Moreover, there is still a gap of 0.65% in accuracy and 0.89% in F1 score between GraphFVD-RC-local_attention and GraphFVD-RC. This highlights the importance of local attention in assigning varying weights to different token vectors based on their importance for each node when learning the node embedding representation of PrVCs. Additionally, GraphFVD-RC-local_attention-global_attention achieves the worst performance on the GITA dataset, with an accuracy of 90.89% and an F1 score of 79.17%. This phenomenon demonstrates that our proposed global attention enables GraphFVD to learn the importance of nodes in PrVCs, leading to a more effective graph-level embedding representation and, consequently, more accurate vulnerability detection. Additionally, the ablation results of GraphFVD across different projects are provided in the Appendix.

6. Discussion and Limitation

GraphFVD proposes a novel code representation, known as PrVC, extracted from CPG. As highlighted in Table 5, PrVC outperforms other code representations, including function-level representations and sequence-level program slices. This demonstrates the effectiveness of our property graph-based program slicing method for fine-grained vulnerability detection. Furthermore, PrVC is represented as a directed multi-relational graph, and we employ a Hierarchical Attention Graph Convolutional Network (HierGCN) to learn its graph embeddings. HierGCN introduces hierarchical attention mechanisms, including local attention and global attention. By leveraging this attention structure, the model effectively integrates the features of different tokens and nodes based on their relative importance, resulting in a robust graph embedding representation. Additionally, HierGCN incorporates residual connections to link multiple RGCN layers, further enhancing its learning capability.

Despite its strengths, our method has several limitations. First, our experiments are limited to C/C++ code, which may necessitate adaptations to effectively detect vulnerabilities in other programming languages. Second, during the automatic generation of ground-truth labels, labeling errors may occur, potentially impacting the model’s ability to accurately identify vulnerabilities. Addressing this issue will require future research to focus on developing more precise methods for automatically annotating vulnerability data, thereby improving the reliability of ground-truth labels and enhancing model performance. Lastly, further investigation is needed to improve the interpretability of deep learning-based models for vulnerability detection. Gaining a deeper understanding of their decision-making processes could provide valuable insights, increase trust in their outputs, and guide future advancements in the field.

7. Conclusion

In this paper, we propose GraphFVD, a novel fine-grained vulnerability detection method. GraphFVD extracts property graph-based slices from the Code Property Graph and utilizes a Hierarchical Attention Graph Convolutional Network to learn graph embeddings, effectively capturing comprehensive vulnerability semantics. We evaluate GraphFVD on real-world vulnerability datasets, with experimental results demonstrating its superior performance compared to state-of-the-art vulnerability detection methods and LLMs. In future work, we plan to explore more effective code representations and develop techniques for locating vulnerable code lines within vulnerable functions. Furthermore, we aim to enhance the performance of Large Language Models (LLMs) in vulnerability detection by incorporating domain knowledge to enrich their contextual understanding.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant no. 61872107).

References

- [1] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, *IEEE Transactions on Dependable and Secure Computing* 19 (4) (2021) 2244–2258. doi:<https://doi.org/10.1109/TDSC.2021.3051525>.
- [2] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, H. Jin, VulDeeLocator: a deep learning-based fine-grained vulnerability detector, *IEEE Transactions on Dependable and Secure Computing* 19 (4) (2021) 2821–2837. doi:<https://doi.org/10.1109/TDSC.2021.3076142>.
- [3] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, M. McConley, Automated vulnerability detection in source code using deep representation learning, in: 2018 17th IEEE international conference on machine learning and applications (ICMLA), IEEE, 2018, pp. 757–762. doi:<https://doi.org/10.1109/ICMLA.2018.00120>.
- [4] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, *Advances in neural information processing systems* 32 (2019). doi:<https://doi.org/10.48550/arXiv.1909.03496>.
- [5] H. Wang, G. Ye, Z. Tang, S. H. Tan, S. Huang, D. Fang, Y. Feng, L. Bian, Z. Wang, Combining graph-based learning with automated data collection for code vulnerability detection, *IEEE Transactions on Information Forensics and Security* 16 (2020) 1943–1958. doi:<https://doi.org/10.1109/TIFS.2020.3044773>.
- [6] S. Cao, X. Sun, L. Bo, Y. Wei, B. Li, Bgnn4vd: Constructing bidirectional graph neural-network for vulnerability detection, *Information and Software Technology* 136 (2021) 106576. doi:<https://doi.org/10.1016/j.infsof.2021.106576>.
- [7] K. Filus, J. Domańska, Software vulnerabilities in tensorflow-based deep learning applications, *Computers & Security* 124 (2023) 102948. doi:<https://doi.org/10.1016/j.cose.2022.102948>.
- [8] W. Guo, Y. Fang, C. Huang, H. Ou, C. Lin, Y. Guo, HyVulDect: A hybrid semantic vulnerability mining system based on graph neural network, *Computers & Security* (2022) 102823. doi:<https://doi.org/10.1016/j.cose.2022.102823>.
- [9] S. Salimi, M. Kharrazi, VulSlicer: Vulnerability detection through code slicing, *Journal of Systems and Software* 193 (2022) 111450. doi:<https://doi.org/10.1016/j.jss.2022.111450>.
- [10] Z. Tang, Q. Hu, Y. Hu, W. Kuang, J. Chen, SEVulDet: A semantics-enhanced learnable vulnerability detector, in: 2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), IEEE, 2022, pp. 150–162. doi:<https://doi.org/10.1109/DSN53405.2022.00026>.
- [11] F. Yamaguchi, N. Golde, D. Arp, K. Rieck, Modeling and discovering vulnerabilities with code property graphs, in: 2014 IEEE Symposium on Security and Privacy, IEEE, 2014, pp. 590–604. doi:<https://doi.org/10.1109/SP.2014.44>.
- [12] H. Liang, L. Wang, D. Wu, J. Xu, MLSA: A static bugs analysis tool based on llvm ir, in: 2016 17th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD), 2016, pp. 407–412. doi:<https://doi.org/10.1109/SNPD.2016.7515932>.
- [13] J. Thomé, L. K. Shar, D. Bianculli, L. Briand, Search-driven string constraint solving for vulnerability detection, in: 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE), IEEE, 2017, pp. 198–208. doi:<https://doi.org/10.1109/ICSE.2017.26>.
- [14] Flawfinder, <https://dwheeler.com/flawfinder/>, Accessed: Aug. 14, 2024.
- [15] Checkmarx, <https://www.checkmarx.com/>, Accessed: Aug. 14, 2024.
- [16] Z. Li, D. Zou, S. Xu, H. Jin, H. Qi, J. Hu, VulPecker: an automated vulnerability detection system based on code similarity analysis, in: Proceedings of the 32nd annual conference on computer security applications, 2016, pp. 201–213. doi:<https://doi.org/10.1145/2991079.2991102>.
- [17] H. Sun, L. Cui, L. Li, Z. Ding, Z. Hao, J. Cui, P. Liu, VDSimilar: Vulnerability detection based on code similarity of vulnerabilities and patches, *Computers & Security* 110 (2021) 102417. doi:<https://doi.org/10.1016/j.cose.2021.102417>.
- [18] W. Wang, G. Li, S. Shen, X. Xia, Z. Jin, Modular tree network for source code representation learning, *ACM Transactions on Software Engineering and Methodology (TOSEM)* 29 (4) (2020) 1–23. doi:<https://doi.org/10.1145/3409331>.
- [19] B. Steenhoeck, H. Gao, W. Le, Dataflow analysis-inspired deep learning for efficient vulnerability detection, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, 2024, pp. 1–13.
- [20] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, *Network and Distributed Systems Security (NDSS) Symposium* 2018 (2018). doi:<https://doi.org/10.14722/ndss.2018.23158>.
- [21] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, C. Tao, Mvd: memory-related vulnerability detection based on flow-sensitive graph neural networks, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 1456–1468. doi:<https://doi.org/10.1145/3510003.3510219>.
- [22] Y. Dong, Y. Tang, X. Cheng, Y. Yang, S. Wang, SedSVD: Statement-level software vulnerability detection based on relational graph convolutional network with subgraph embedding, *Information and Software Technology* 158 (2023) 107168. doi:<https://doi.org/10.1016/j.infsof.2023.107168>.
- [23] D. Zou, S. Wang, S. Xu, Z. Li, H. Jin, μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection, *IEEE Transactions on Dependable and Secure Computing* 18 (5) (2019) 2224–2236. doi:<https://doi.org/10.1109/TDSC.2019.2942930>.
- [24] Y. Mirsky, G. Macon, M. Brown, C. Yagemann, M. Pruett, E. Downing, S. Mertoguno, W. Lee, {VulChecker}: Graph-based vulnerability localization in source code, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 6557–6574.
- [25] V.-A. Nguyen, D. Q. Nguyen, V. Nguyen, T. Le, Q. H. Tran, D. Phung, ReGVD: Revisiting graph neural networks for vulnerability detection, in: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, 2022, pp. 178–182. doi:<https://doi.org/10.1145/3510454.3516865>.
- [26] X. Cheng, G. Zhang, H. Wang, Y. Sui, Path-sensitive code embedding via contrastive learning for software vulnerability detection, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 519–531. doi:<https://doi.org/10.1145/3533767.3534371>.
- [27] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, H. Jin, VulCNN: An image-inspired scalable vulnerability detection system, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 2365–2376. doi:<https://doi.org/10.1145/3510003.3510229>.
- [28] M. Fu, C. Tantithamthavorn, Linevul: A transformer-based line-level vulnerability prediction, in: Proceedings of the 19th International Conference on Mining Software Repositories, 2022, pp. 608–620.

- [29] B. Yuan, Y. Lu, Y. Fang, Y. Wu, D. Zou, Z. Li, Z. Li, H. Jin, Enhancing deep learning-based vulnerability detection by building behavior graph model, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, 2023, pp. 2262–2274.
- [30] W. Wang, T. N. Nguyen, S. Wang, Y. Li, J. Zhang, A. Yadavally, Deepvd: Toward class-separation features for neural network vulnerability detection, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, 2023, pp. 2249–2261.
- [31] Z. Tian, B. Tian, J. Lv, Y. Chen, L. Chen, Enhancing vulnerability detection via ast decomposition and neural sub-tree encoding, Expert Systems with Applications 238 (2024) 121865.
- [32] C. Zhang, H. Liu, J. Zeng, K. Yang, Y. Li, H. Li, Prompt-enhanced software vulnerability detection using chatgpt, in: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, 2024, pp. 276–277.
- [33] G. Lu, X. Ju, X. Chen, W. Pei, Z. Cai, Grace: Empowering llm-based software vulnerability detection with graph structure and in-context learning, Journal of Systems and Software 212 (2024) 112031.
- [34] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, L. Shujie, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, et al., Graphcodebert: Pre-training code representations with data flow, in: International Conference on Learning Representations, 2020. doi:<https://doi.org/10.48550/arXiv.2009.08366>.
- [35] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. Van Den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: The Semantic Web: 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, Proceedings 15, Springer, 2018, pp. 593–607. doi:https://doi.org/10.1007/978-3-319-93417-4_38.
- [36] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 770–778. doi:<https://doi.org/10.1109/cvpr.2016.90>.
- [37] NIST, Nist software assurance reference dataset, <https://samate.nist.gov/SRD/index.php>, Accessed: Nov. 15, 2024.
- [38] NIST, National vulnerability database, <https://nvd.nist.gov/>, Accessed: Nov. 15, 2024.
- [39] Y. Zheng, S. Pujar, B. Lewis, L. Buratti, E. Epstein, B. Yang, J. Laredo, A. Morari, Z. Su, D2a: A dataset built for ai-based vulnerability detection methods using differential analysis, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), IEEE, 2021, pp. 111–120. doi:<https://doi.org/10.1109/ICSE-SEIP52600.2021.00020>.
- [40] Q. Zhu, D. Guo, Z. Shao, D. Yang, P. Wang, R. Xu, Y. Wu, Y. Li, H. Gao, S. Ma, et al., Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence, arXiv preprint arXiv:2406.11931 (2024).
- [41] J. D. M.-W. C. Kenton, L. K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of NAACL-HLT, 2019, pp. 4171–4186. doi:<http://dx.doi.org/10.18653/v1/N19-1423>.
- [42] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, et al., Codebert: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 1536–1547. doi:<http://dx.doi.org/10.18653/v1/2020.findings-emnlp.139>.

Appendix

Table .8: Ablation results across different projects.

Project	Model	ACC	F1	P	FNR	FPR
OpenSSL	GraphFVD	91.81	85.50	85.86	14.94	5.58
	GraphFVD-RC	91.44	85.39	86.79	16.04	5.45
	GraphFVD-RC-local_attention	91.09	85.00	85.25	15.32	6.24
	GraphFVD-RC-local_attention-global_attention	90.99	84.68	85.71	16.40	5.94
Libav	GraphFVD	94.00	77.93	79.36	23.53	3.21
	GraphFVD-RC	93.55	77.20	78.50	24.13	3.51
	GraphFVD-RC-local_attention	93.41	76.26	79.20	26.54	3.26
	GraphFVD-RC-local_attention-global_attention	92.97	74.87	77.13	27.32	3.64
FFmpeg	GraphFVD	91.17	80.62	82.13	20.85	5.21
	GraphFVD-RC	90.84	79.39	80.71	21.90	5.45
	GraphFVD-RC-local_attention	90.60	79.33	80.75	22.07	5.60
	GraphFVD-RC-local_attention-global_attention	90.16	78.21	80.18	23.69	5.68
Linux Kernel	GraphFVD	91.35	81.89	80.32	16.59	6.29
	GraphFVD-RC	91.02	81.13	79.84	17.65	6.40
	GraphFVD-RC-local_attention	89.12	78.98	74.35	15.89	9.34
	GraphFVD-RC-local_attention-global_attention	89.45	78.92	76.60	18.73	8.00